



Introduction to 8086 Microprocessor

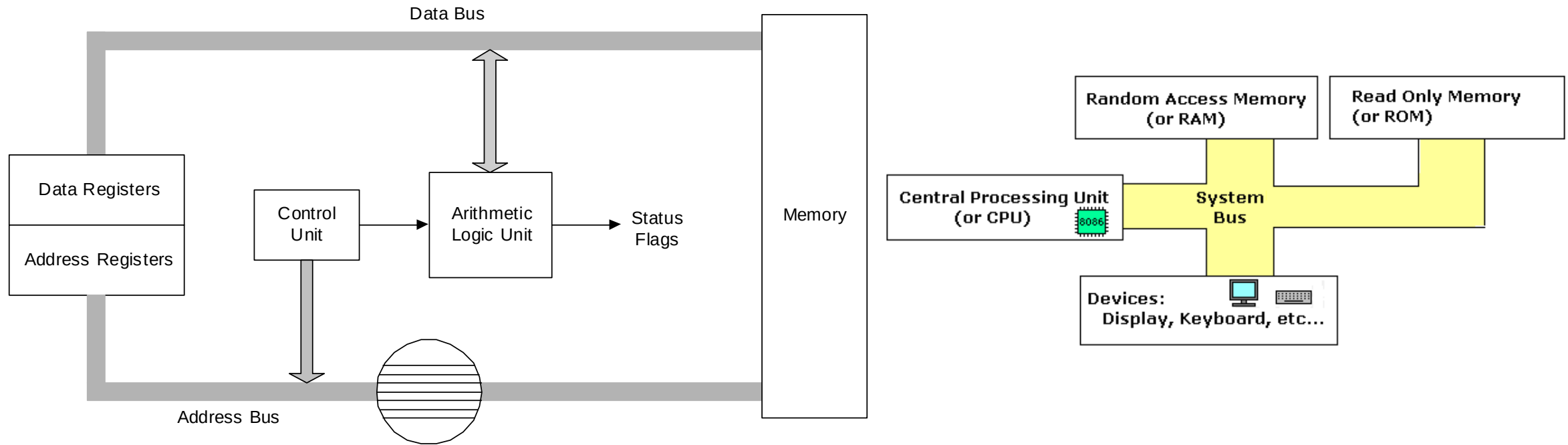
Chapter 1

LEARNING OUTCOMES

UPON COMPLETING THIS CHAPTER THE READER WILL KNOW THE ARCHITECTURE OF 8086 WITH SPECIAL ATTENTION TO THE FOLLOWINGS:

- THE ESSENTIAL FUNCTIONAL COMPONENTS OF 8086
- GENERAL REGISTERS AND THEIR FUNCTIONS
- SPECIAL REGISTERS AND THEIR FUNCTIONS
- FLAG REGISTER CONTAINING STATUS FLAGS AND THEIR FUNCTIONS
- ADDRESSING MECHANISM AND MEMORY SEGMENTATION IN 8086
- PHYSICAL AND LOGICAL ADDRESSING
- PIPELINING
- INTERRUPT STRUCTURE AND THEIR FUNCTIONALITY

Simplified CPU Design



8086 Features

16-bit Arithmetic Logic Unit

16-bit data bus (8088 has 8-bit data bus)

20-bit address bus - $2^{20} = 1,048,576 = 1 \text{ M}$

Follows CISC architecture

The address refers to a byte in memory. In the 8088, these bytes come in on the 8-bit data bus.

- In the 8086, bytes at even addresses come in on the low half of the data bus (bits 0-7) and bytes at odd addresses come in on the upper half of the data bus (bits 8-15)..

The 8086 can read a 16-bit word at an even address in one operation and at an odd address in two operations. The 8088 needs two operations in either case.

The least significant byte of a word on an 8086 family microprocessor is at the lower address. This is known as Little Endian Format

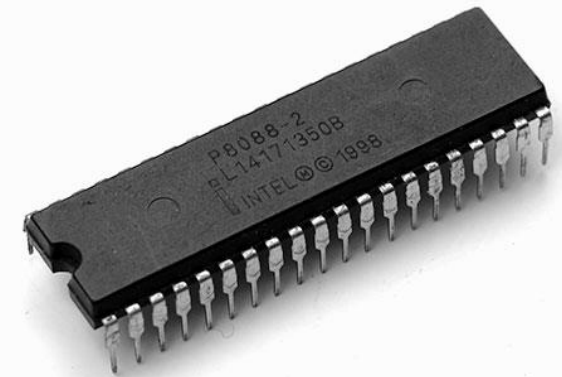
Intel 8086 Microprocessors

8086 consists of two internal units

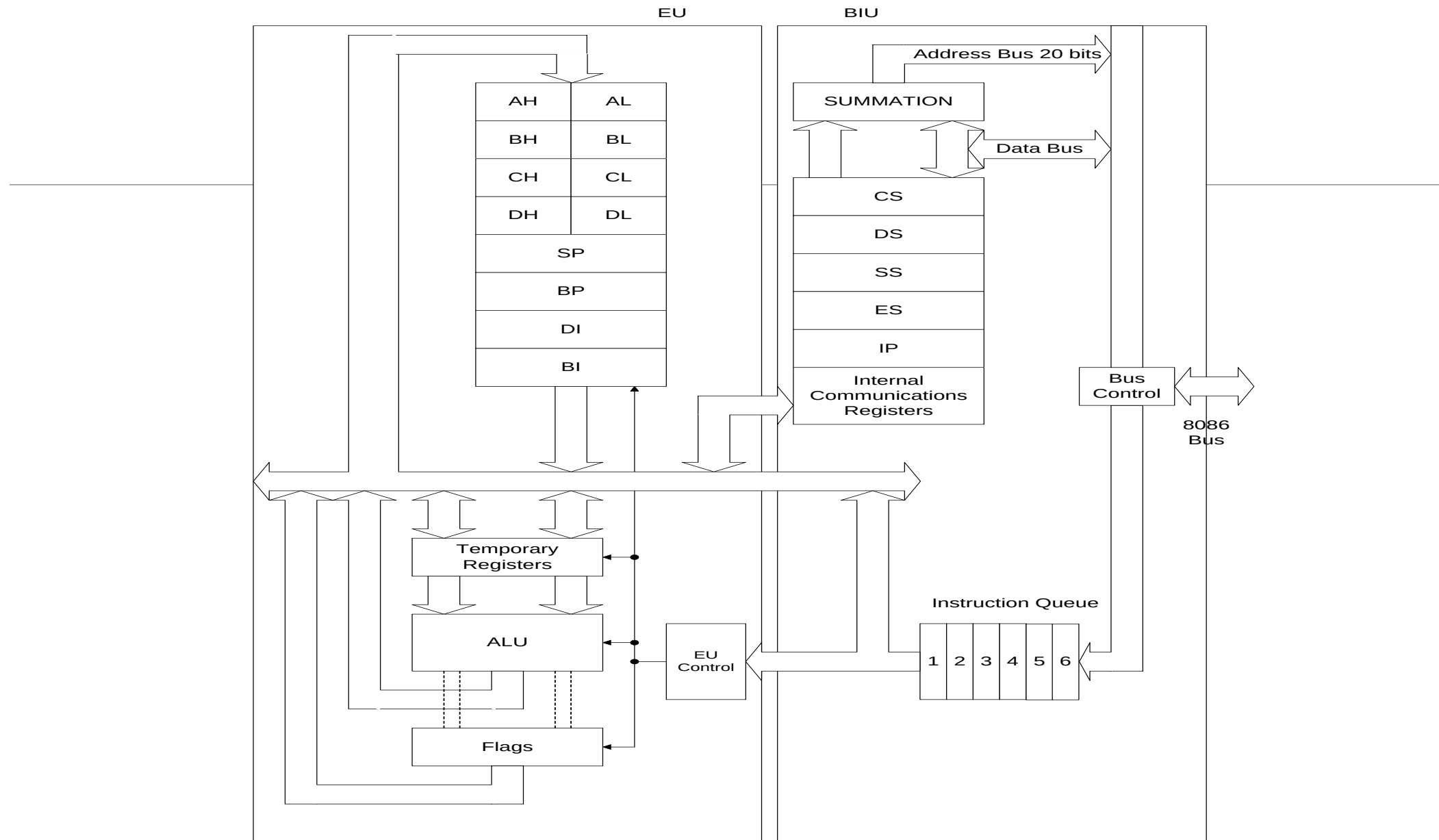
- The execution unit (EU) - executes the instructions
- The bus interface unit (BIU) - fetches instructions, reads operands and writes results

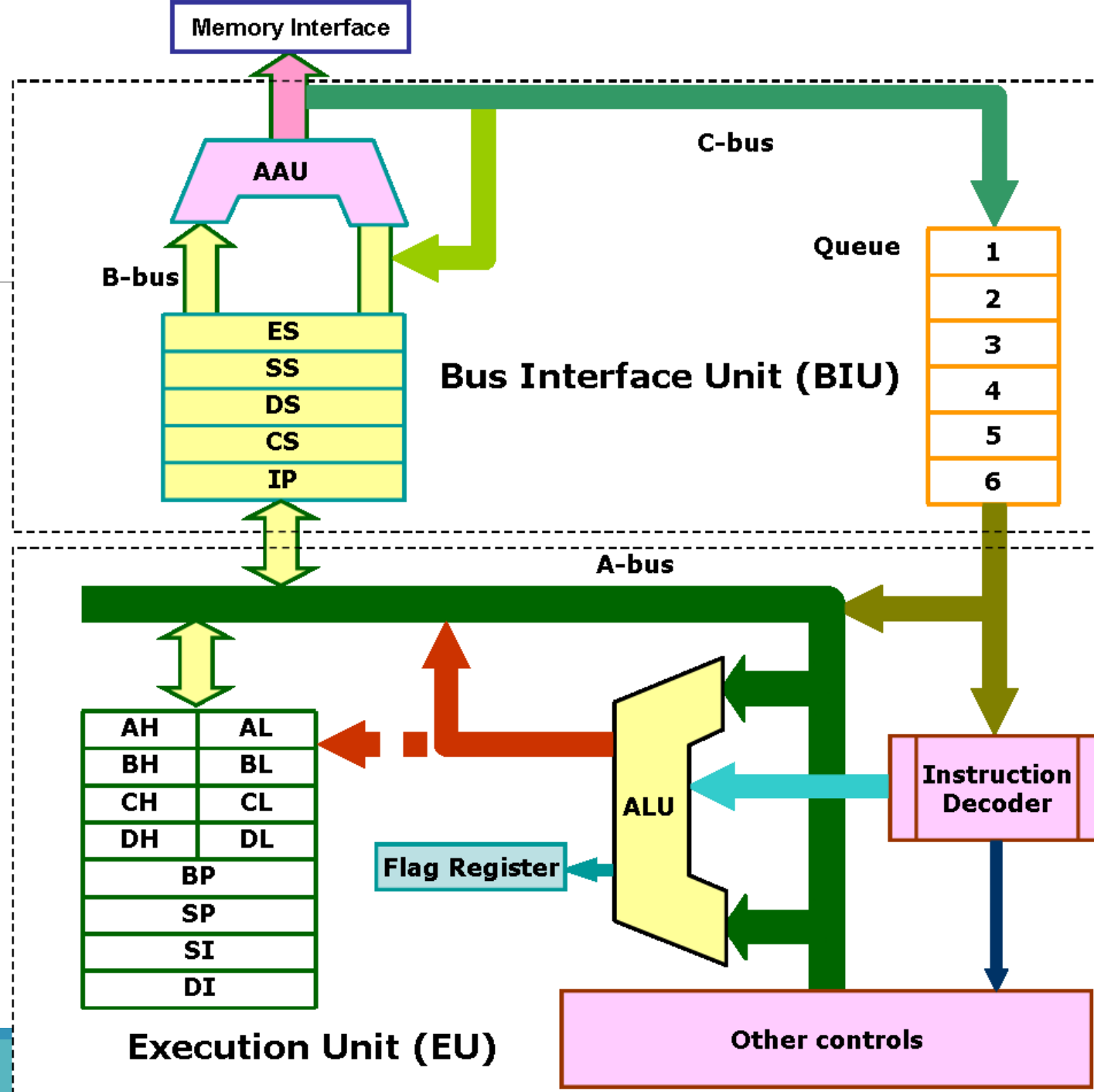
The 8086 has a 6B prefetch queue

The 8088 has a 4B prefetch queue



8086 Internal Organisation





8086 Architecture - BIU

The BIU contains the following registers:

IP - the Instruction Pointer

CS - the Code Segment Register

DS - the Data Segment Register

SS - the Stack Segment Register

ES - the Extra Segment Register

The BIU fetches instructions using the CS and IP, written CS:IP, to construct the 20-bit address.

Data is fetched using a segment register (usually the DS) and an effective address (EA) computed by the EU depending on the addressing mode.

8086 Architecture - EU

The EU contains the following 16-bit registers:

- **AX - the Accumulator**
- **BX - the Base Register**
- **CX - the Count Register**
- **DX - the Data Register**
- **SP - the Stack Pointer** \ defaults to stack segment
- **BP - the Base Pointer** /
- **SI - the Source Index Register**
- **DI - the Destination Register**

These are referred to as general-purpose registers, although they often have a special-purpose use for some instructions.

The AX, BX, CX, and DX registers can be considered as two 8-bit registers, a High byte and a Low byte. This allows byte operations and compatibility with the previous generation of 8-bit processors, the 8080 and 8085. The 8-bit registers are:

- AX --> AH,AL
- BX --> BH,BL
- CX --> CH,CL
- DX --> DH,DL

Register Memories

Registers are in the CPU and are referred to by specific names

-
- **Data registers**
 - Hold data for an operation to be performed
 - There are 4 data registers (AX, BX, CX, DX)
 - **Address registers**
 - Hold the address of an instruction or data element
 - **Segment registers** (CS, DS, ES, SS)
 - **Pointer registers** (SP, BP, IP)
 - **Index registers** (SI, DI)
 - **Status register**
 - Keeps the current status of the processor
 - On an IBM PC the status register is called the FLAGS register
 - In total there are fourteen 16-bit registers in an 8086/8088

Data Registers: AX, BX, CX, DX

Instructions execute faster if the data is in a register.

Low and High bytes of the data registers can be accessed separately

- AH, BH, CH, DH are the high bytes
- AL, BL, CL, and DL are the low bytes

Data Registers are general purpose registers but they also perform special functions

AX

- Accumulator Register
- Preferred register to use in arithmetic, logic and data transfer instructions because it generates the shortest Machine Language Code
- Must be used in multiplication and division operations
- Must also be used in I/O operations

Data Registers: AX, BX, CX, DX

BX

- Base Register
- Also serves as an address register
- Used in array operations
- Used in Table Lookup operations (XLAT)

CX

- Count register
- Used as a loop counter
- Used in shift and rotate operations

DX

- Data register
- Used in multiplication and division
- Also used in I/O operations

Pointer and Index Registers

Contain the offset addresses of memory locations

Can also be used in arithmetic and other operations

SP: Stack pointer

- Used with SS to access the stack segment

BP: Base Pointer

- Primarily used to access data on the stack
- Can be used to access data in other segments

SI: Source Index register

- is required for some string operations
- When string operations are performed, the SI register points to memory locations in the data segment which is addressed by the DS register. Thus, SI is associated with the DS in string operations.

DI: Destination Index register

- is also required for some string operations.
- When string operations are performed, the DI register points to memory locations in the data segment which is addressed by the ES register. Thus, DI is associated with the ES in string operations.

The SI and the DI registers may also be used to access data stored in arrays

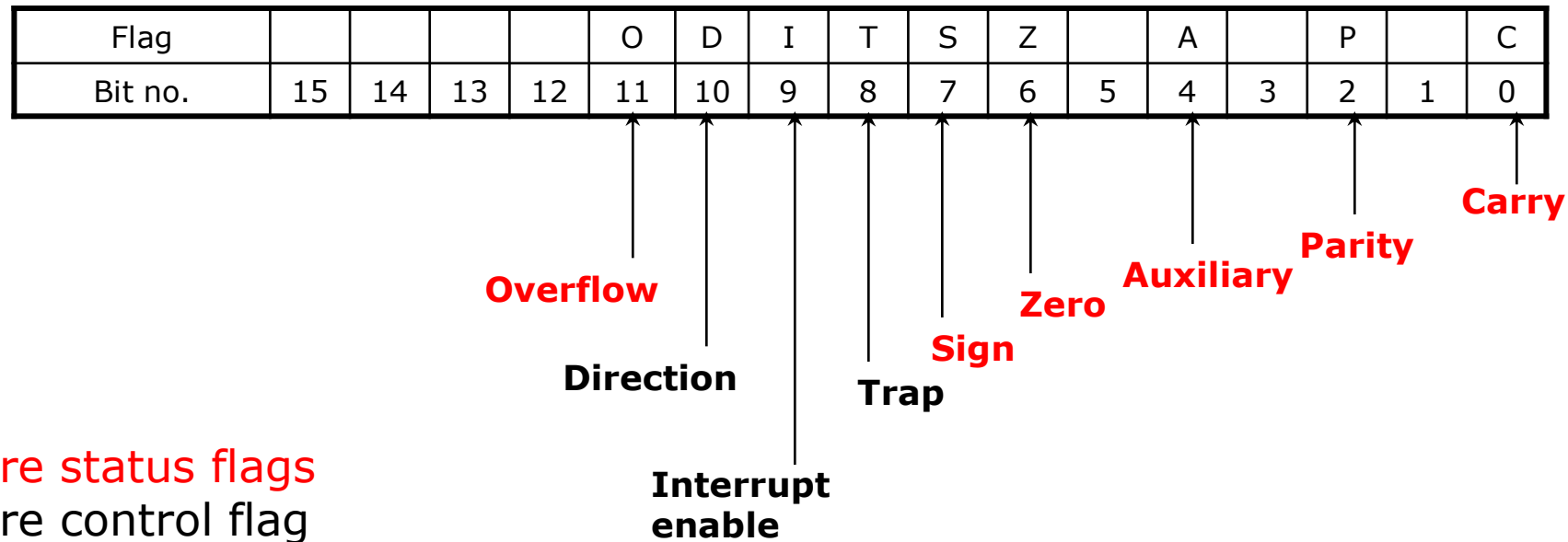
Flag Register

Conditional flags:

- They are set according to some results of arithmetic operation. You do not need to alter the value yourself.

Control flags:

- Used to control some operations of the MPU. These flags are to be set by you in order to achieve some specific purposes.



1. **CF (carry)** Contains carry from leftmost bit following arithmetic, also contains last bit from a shift or rotate operation.
2. **OF (overflow)** Indicates overflow of the leftmost bit during arithmetic.
3. **SF (sign)** Contains the resulting sign of an arithmetic operation (1=negative)
4. **ZF (zero)** Indicates when the result of arithmetic or a comparison is zero. (1=yes)
5. **AF (auxiliary carry)** Contains carry out of bit 3 into bit 4 for specialized arithmetic.
6. **PF (parity)** Indicates the number of 1 bits that result from an operation.
7. **DF (direction)** Indicates left or right for moving or comparing string data.
8. **IF (interrupt)** Indicates whether external interrupts are being processed or ignored.
9. **TF (trace)** Permits operation of the processor in single step mode

Segment Registers - CS, DS, SS & ES

Are Address registers

Store the memory addresses of instructions and data

Memory Organization

- Each byte in memory has a 20 bit address starting with 0 to $2^{20}-1$ or 1 meg of addressable memory
- Addresses are expressed as 5 hex digits from 00000 - FFFFF
- Problem: But 20 bit addresses are *TOO BIG* to fit in 16 bit registers!
- Solution: Memory Segment
 - Block of 64K (65,536) consecutive memory bytes
 - A segment number is a 16 bit number
 - Segment numbers range from 0000 to FFFF
 - Within a segment, a particular memory location is specified with an offset
 - An offset also ranges from 0000 to FFFF

Segmented Memory

Segment addresses 64K of memory

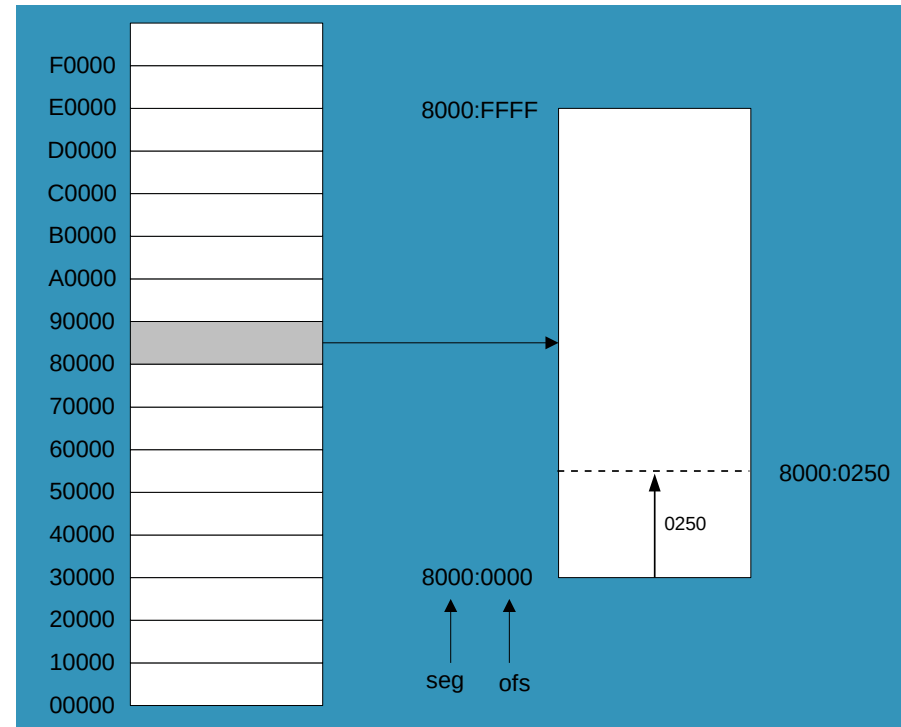
A segmented register contains the starting address of a segment

- The absolute address can be determined by appending a hexadecimal zero

A offset is the distance from the beginning of the segment to a particular instruction or variable.

Segmented memory addressing:

- absolute (linear) address is a combination of a 16-bit segment value added to a 16-bit offset

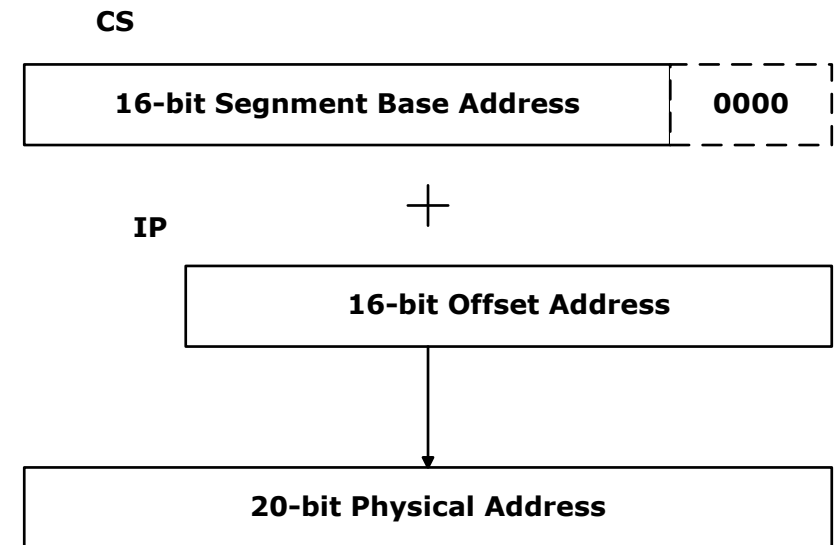


8086 20-bit Addresses

Logical Address is specified as **segment: offset**

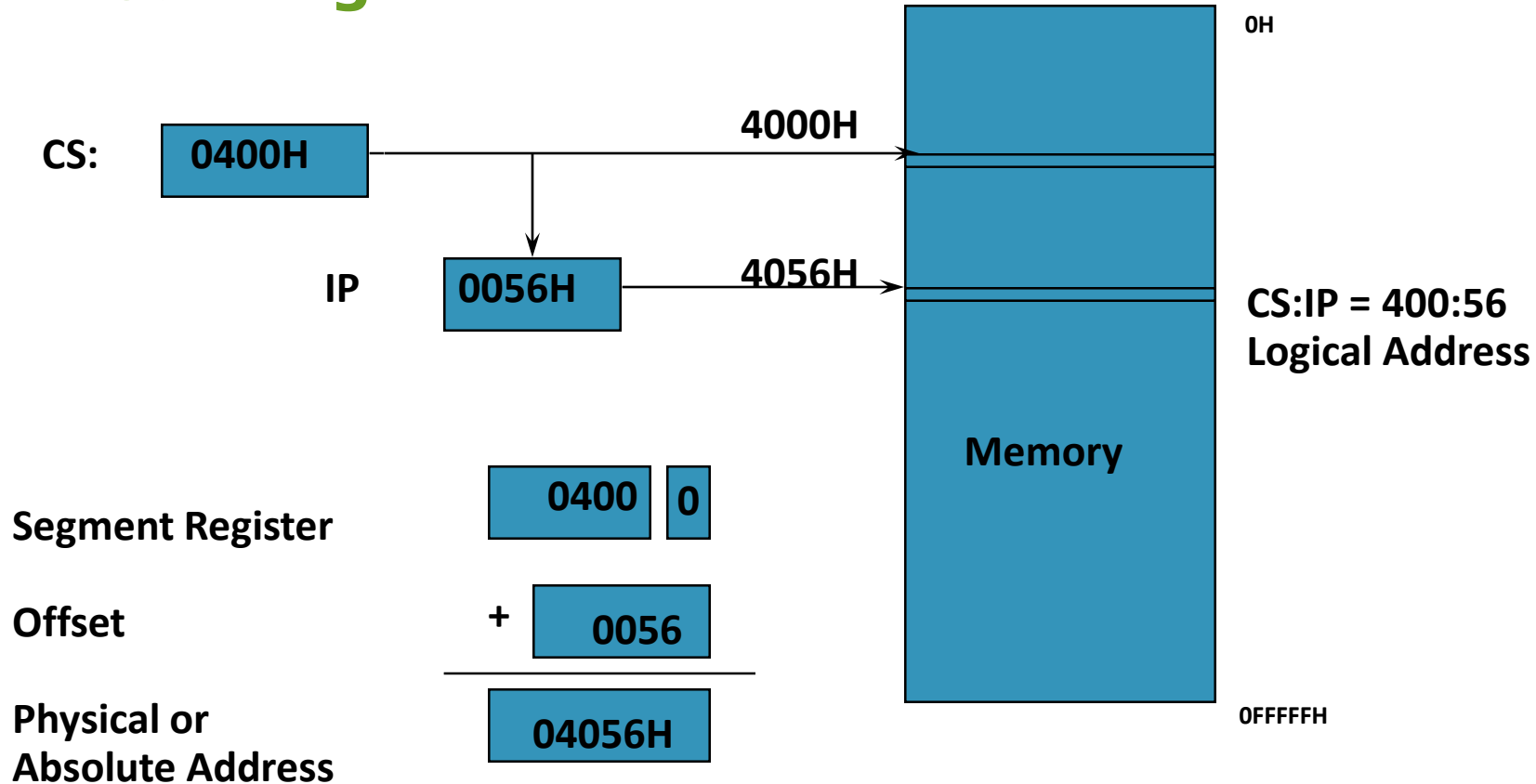
Physical address is obtained by shifting the segment address 4 bits to the left and adding the offset address

Thus the physical address of the logical address A4FB:4872 is

$$\begin{array}{r} \text{A4FB0} \\ + \text{4872} \\ \hline \text{A9822} \end{array}$$


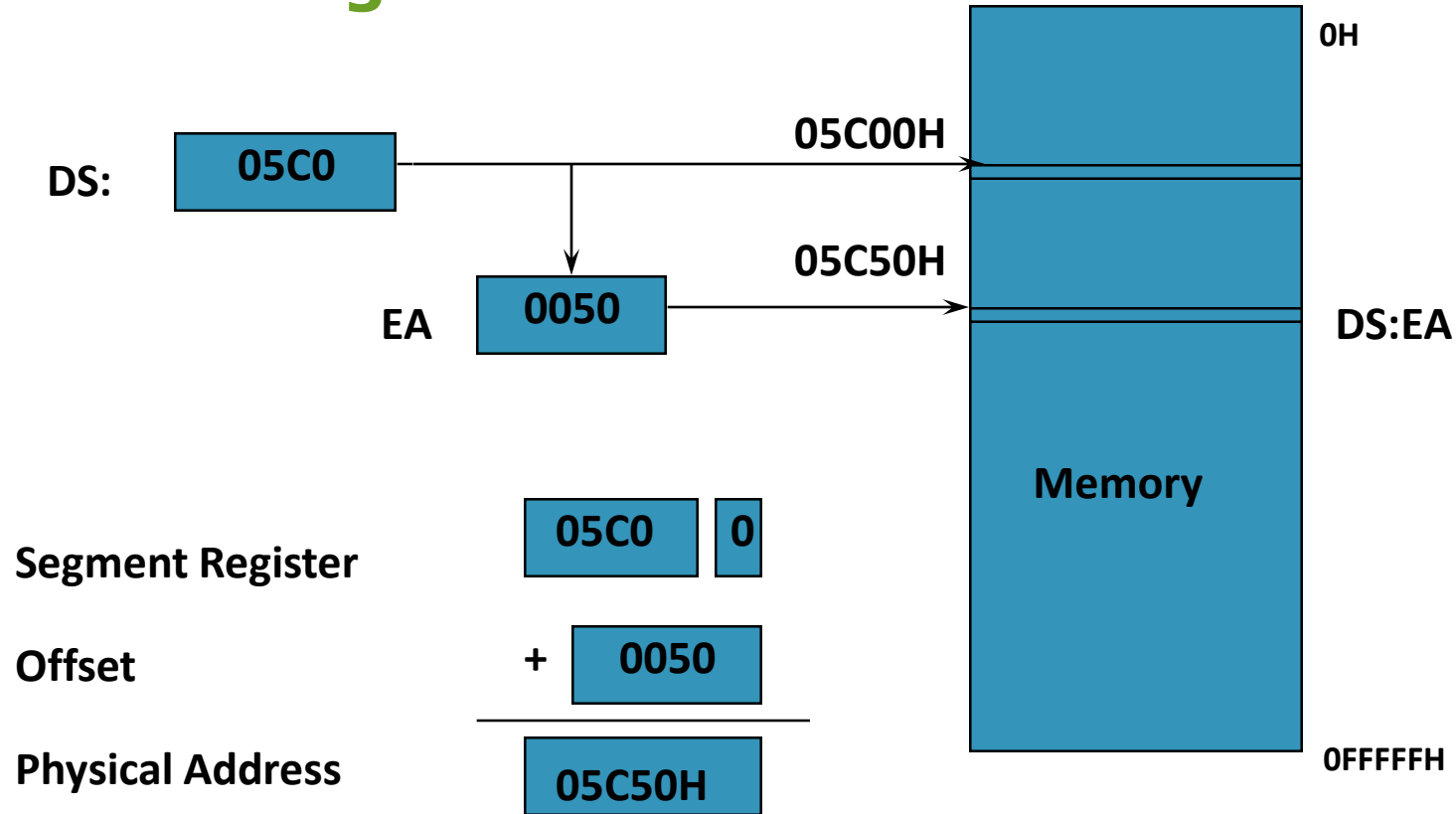
The physical address is also called the absolute address.

Code Segment



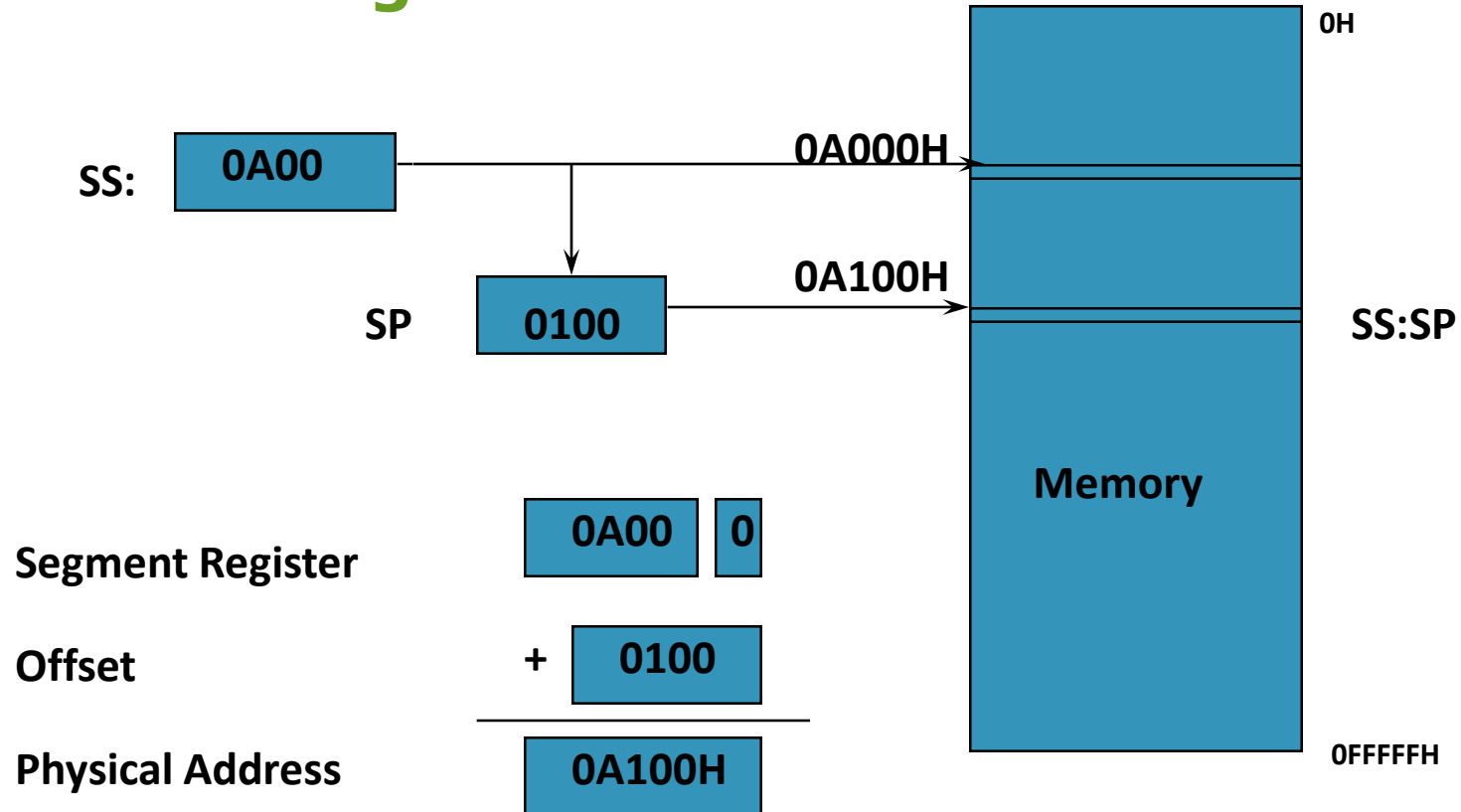
The offset is the distance in bytes from the start of the segment.
The offset is given by the IP for the Code Segment.
Instructions are always fetched with using the CS register.

Data Segment



Data is usually fetched with respect to the DS register.
The effective address (EA) is the offset.
The EA depends on the addressing mode.

Stack Segment



The offset is given by the SP register.

The stack is always referenced with respect to the stack segment register.

The stack grows toward decreasing memory locations.

The SP points to the last or top item on the stack.

PUSH - pre-decrement the SP

POP - post-increment the SP

8086 Interrupts

Interrupt operations fall into two classes; software or hardware initiated. The software initiated interrupts and software aspects of hardware interrupts are specified in the Instruction Set description. Hardware interrupts can be classified as non-Maskable or Maskable.

Interrupt Pointer Table.

Interrupt Type Code		20-bit Memory Address	
0	IP	00001H	00000H
	CS	00003H	00002H
1	IP	00005H	00004H
	CS	00007H	00006H
2	IP	00009H	00008H
	CS	0000BH	0000AH
.	.	.	.
.	.	.	.
.	.	.	.
255	IP	003FDH	003FCH
	CS	003FFH	003FEH

The 8086 loads the values of IP and CS from the 20-bit physical address 00008H and 0000AH from the pointer table. The user must store the desired 16-bit values of IP and CS in these locations.

For example if the NMI service routine is in location CS:IP = 4000H:0100H, the user should store 4000H in location 00008H and 0100H in location 0000AH.

In response to an interrupt the 8086 does the following tasks:

1. Pushes the Flag register (PSW), CS and IP onto the stack
2. Clears the TF and IF flags
3. Loads the IP and CS from the pointer table using the type code

An Interrupt Service Routine (ISR) should always be terminated with the IRET (Interrupt Return) instruction. With the IRET instruction 8086 pops the top three stack words into IP, CS and PSW (flags), thus returning to the right place in the main program.

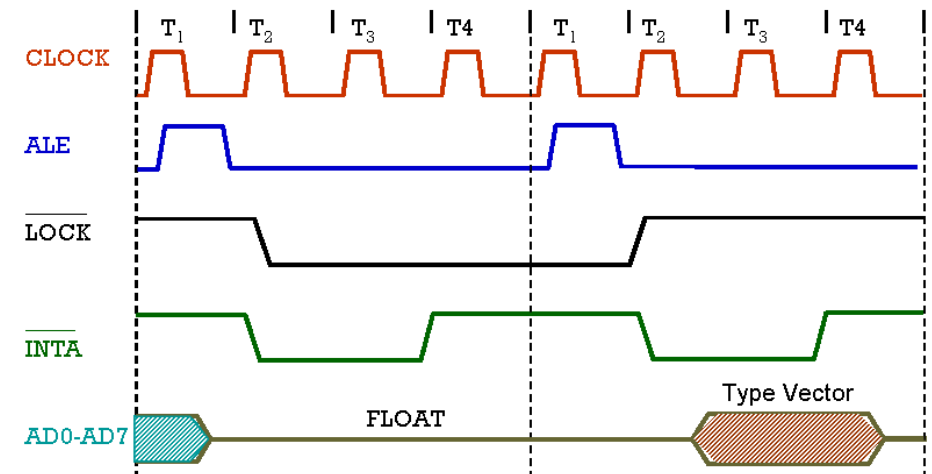
Interrupt Type Codes

The 256 interrupt type codes are assigned as follows:

1. Predefined Interrupts (Types 0 to 4), 5 interrupts
2. Reserved Interrupts (Types 5 to 31), 27 interrupts
3. User defined Maskable Interrupts (Types 32 to 255), 224 interrupts

Predefined interrupts and vector locations.

Interrupt Name	Type	Vector Address (CS:IP)
Divide by Zero Interrupt	Type 0	0002H:0000H
Single Step Interrupt	Type 1	0006H:0004H
NMI	Type 2	000AH:0008H
Breakpoint Interrupt	Type 3	000EH:000CH
Overflow Interrupt	Type 4	0012H:0010H



Thank You

A solid teal-colored horizontal bar at the bottom of the slide, with a slight gradient from left to right.